



Vole

Local-first usage, cost and reliability monitor for AI coding agents — complete documentation

VERSION	0.1.0
BUILT	2026-09-04
REPOSITORY	github.com/jayimf432/vole
PLATFORM	macOS 26.5 (arm64)
LICENCE	MIT

Contents

01	Overview	What Vole is, what is real vs estimated, and how to run it.
02	How it was built	The investigation, the five bugs that shaped the architecture, and what was deliberately not built.
03	Architecture	Module map, database schema, idempotency contract, and the design system.
04	Data sources	What each tool writes to disk, with real samples and the traps in each format.
05	Development	Setup, commands, layering rules, testing philosophy, and debugging recipes.
06	Extending	Complete recipes: add a collector, a rule, a model rate, or a dashboard panel.
07	Roadmap	Ranked next steps, good first issues, and the principles to preserve.

Vole

Local-first usage, cost and reliability monitor for AI coding agents.

Vole reads the logs your AI coding tools already write to disk, normalises them into one schema, and surfaces both spend and *misbehaviour* — runaway tool loops, token-burn spikes, retry storms, rate-limit pressure.

It is a monitoring tool, not a cost tracker: the incident feed is the point.

Everything runs locally. No scraping, no cloud APIs, no logging into any web UI.

Contents

- [Why this exists](#)
- [What is real, and what is not](#)
- [The duplication trap](#)
- [Quick start](#)
- [The interface](#)
- [Command line and integrations](#)
- [Anomaly rules](#)
- [Cost model](#)
- [Verifying it works](#)
- [Architecture](#)
- [Known limitations](#)
- [Documentation](#)

Why this exists

Developers now run several AI coding agents side by side. Each burns tokens independently, and none of them tells you when one has gone wrong — stuck in a tool loop, retrying against a broken API, or quietly re-reading the same 200K-token context forty times.

Existing tools answer *"how much did I spend?"*. Vole answers *"is anything going wrong right now?"* — and shows you the spend as a side effect.

What is real, and what is not

This is the most important section in this README. Different tools expose wildly different data, and Vole never papers over the difference.

TOOL	SOURCE ON DISK	TOKENS	COST	BADGE
Claude Code	~/.claude/projects/**/*.*.jsonl (sessions and their subagents)	Exact	Exact	exact
Codex CLI	~/.codex/sessions/**/rollout-*.jsonl	Exact	unavailable	exact
OpenCode	~/.local/share/opencode/opencode.db (message table)	Exact	Exact (OpenCode's own, per provider)	exact
Grok CLI	~/.grok/logs/unified.jsonl (shell.turn.inference_done)	Exact	unavailable (no xAI rate)	exact
Devin	~/Library/Application Support/Devin/User/acp-messages/*.*.db	None recorded	none	no tokens
Cursor	~/.cursor/ai-tracking/ai-code-tracking.db	None recorded	none	no tokens
Antigravity	~/.gemini/antigravity-ide/brain/<id>/	None recorded	none	no tokens

Why each one is what it is

Claude Code — exact. Its JSONL transcripts record per-response `input_tokens` , `output_tokens` , `cache_read_input_tokens` , and cache creation already split by TTL (`ephemeral_5m` / `ephemeral_1h`). That split is what lets Vole compute cost *exactly* rather than approximately. Subagents (the Agent tool, workflows) write their own transcripts under `<project>/<session>/subagents/...` , tagged with the parent session id; Vole walks the tree so that spend is counted too — on the development machine it was 16% of all messages.

Codex — exact tokens, unknown cost. Rollout logs emit `token_count` events carrying both a cumulative `total_token_usage` and a per-turn `last_token_usage` . Vole derives usage from the *delta of the cumulative counter* — summing the cumulative field would double-count catastrophically, and the per-turn field alone over-counts because Codex sometimes emits the same event twice. Cost renders as `-` because no published per-token rate for `gpt-5.1-codex-max` is loaded.

Exact tokens do not imply known cost. The UI keeps those two ideas separate rather than printing `$0` .

Cursor — no token data exists locally. `ai-code-tracking.db` was inspected directly: it has *no token columns at all*. It stores code-hash attribution (which code came from which model/request) to power Cursor's "% AI-written code" feature, and `state.vscdb` holds no usage keys either. Cursor's token accounting lives server-side behind its account, which this project will not touch. Its rows carry real models, sessions and timestamps with `NULL` tokens.

Estimating Cursor tokens from lines of code was considered and **rejected**. It would be a fabricated number wearing an "estimated" badge — exactly the failure mode the confidence system exists to prevent.

Antigravity — activity only. Two hard constraints: it is a flat-rate subscription product that exposes no client-readable per-token usage, and its conversation payloads (`conversations/*.*.pb`) are high-entropy **encrypted** blobs — byte inspection shows no parseable protobuf wire format. The plaintext artifacts under `brain/<id>/` prove a conversation happened, so Vole emits one `activity_only` row per conversation with `NULL` tokens. It does **not** multiply a turn count by a guessed constant — that would be a fabricated number.

OpenCode — exact, cost included. `~/.local/share/opencode/opencode.db` stores every assistant turn as one `message` row whose `data` JSON carries `cost` , a `tokens` object split into `input` / `output` / `reasoning` / `cache read+write`, the model and provider ids, per-turn timestamps and the working directory. Keyed on the message id, so re-reading the table each poll is idempotent. Cost is OpenCode's own figure — it prices every provider it supports, so a free local model shows a genuine `$0` , not an unknown. Tool names come from the `part` table, a

failed provider call (the message carries an `error`) counts as an error, and a subagent's child session is folded into its parent session with `agent_id = <agent>:<child id>`, so a session's spend is one tree.

Grok CLI — exact tokens, cost unknown. `~/grok/logs/unified.jsonl` logs one `shell.turn.inference_done` line per model call with `ctx.{prompt_tokens, cached_prompt_tokens, completion_tokens, reasoning_tokens}` — the OpenAI-style shape. `prompt_tokens` includes the cached part and `completion_tokens` includes reasoning, so `total = prompt_tokens + completion_tokens`, fresh input is `prompt_tokens - cached_prompt_tokens`, and cache-read is `cached_prompt_tokens`. Model and cwd come from each session's `summary.json`. No xAI rate is loaded, so cost is `-`. `shell.tool.exec_done` names each tool the previous call asked for, and `shell.turn.inference_failed` is a call that returned an error with no usage: it is stored as an `activity_only` row with `is_error = 1`, so retry storms on Grok are visible without inventing a token count.

Devin — activity only. Devin's editor stores its ACP conversation under `acp-messages/<uuid>.db`, but the payloads are pure content — no usage, token or model field exists. Devin's agent runs on Cognition's servers and its accounting stays there, like Cursor. Vole emits one `activity_only` row per agent turn, `NULL` tokens. (The editor also bundles the `claude-code` extension; those runs are already covered by the Claude Code collector and not double-counted.)

The two confidence levels

There is no "estimated" tier, by policy. A token count is either measured or it is absent — never guessed.

- **exact** — read verbatim from the tool's own logs. `total_tokens` is a plain sum of real usage fields (or, for Codex, a delta of the tool's own cumulative counter).
- **activity_only** — the call happened; the tool records no tokens locally (Cursor, Antigravity). The token fields are genuinely `NULL`, and these rows are **excluded from every token and cost aggregate** while still counting as calls.

`pnpm verify` independently re-derives every stored Claude Code and OpenCode row from its source and requires an exact match.

The duplication trap

Claude Code writes the **same API response to its transcript more than once**. Measured on real logs: ~18K raw usage records collapse to ~7.5K unique `message.id` values.

Summing rows naively inflates every number by ~2.4x.

Vole keys each event on `claude_code:<message.id>` under a `UNIQUE` constraint, so every re-scan is idempotent and the polling loop is safe by construction.

The copies are **not** always byte-identical: the first is written mid-stream with `output_tokens: 0` and no `stop_reason`, then rewritten complete. So the collector coalesces the occurrences it sees each pass down to the fullest one, and the insert is an upsert that upgrades an already-stored placeholder when a later copy carries more tokens — it never downgrades, so re-reading identical data stays a no-op. `pnpm verify` reconciles every stored row against the fullest copy in the logs and reports the live duplication factor.

Quick start

Vole is two processes: a **collector** (Node) that parses the logs into `~/vole/vole.db`, and a native **macOS app** (SwiftUI) that reads that database. Requires **Node ≥ 22**, **pnpm**, and **Xcode 26** for the app. Built and tested on **macOS 26.5 (arm64)**.

```
git clone https://github.com/launchsafe/vole
cd vole
pnpm install
```

1. Collect

Parses every source into `~/vole/vole.db` and runs the anomaly rules.

```
pnpm collect:once    # one pass, prints a full summary to the console
pnpm collect         # continuous polling every 5s – this is the actual monitor
```

While polling, a **desktop notification** fires for every new `warn / critical` incident whose window is less than 15 minutes old (macOS via `osascript`, Linux via `notify-send`). A first scan over months of history stays silent. Pass `--no-notify` to turn it off. One bad pass (a locked file, a disk hiccup) is logged and the loop carries on.

2. The app

```
pnpm app             # swift run – the menu-bar app, no bundling
pnpm app:bundle       # build/Vole.app, release build, ad-hoc signed, and open it
```

The app lives in the menu bar (no Dock icon until you open the dashboard window) and re-reads the database every few seconds. It needs nothing but the collector: no server, no port, no browser. Details, packaging and the `-dump` headless check are in [apps/mac/README.md](#).

Demo data

Real logs are sparse and never trigger the error-storm or rate-limit rules, so a demo on live data alone shows a thin incident feed.

```
pnpm seed            # 30 days of synthetic history, exercises the rules
pnpm seed:purge       # removes every seeded row
```

Seed rows are written with `source='seed'`; collectors only ever write `source='live'`. The app shows a **Demo Data** badge whenever seed rows exist (it charts live rows only), and purge is a single `DELETE ... WHERE source='seed'`. Live data is never touched.

Detection is partitioned by source, so seeded rows and live rows never share a statistical baseline — demo data cannot redefine what "normal" means for your real usage.

The interface

The menu-bar panel

The menu-bar item shows live tokens for the chosen range (or cost, or just the mark), tinted red or orange while an incident is open in the last hour. Click it for the panel: tokens and equivalent value, a sparkline of the last 24 buckets, per-tool bars, and the range picker. At that size it answers *"is anything wrong, and how hard are we going?"* and nothing else.

If the collector is not running the panel says so, with the command to copy. If it ran but has not checked in recently, a banner says when it was last seen — "quiet" and "collector down" are different states and the app never confuses them.

The dashboard window

The signature element is the **incident-annotated timeline**: tokens stacked per tool in each tool's brand colour, with every incident pinned to the bucket where its rule fired, so a spike and its cause are one glance rather than a chart and a list to correlate by hand. Around it: the KPI row (tokens, equivalent value, calls, sessions, cache hit, errors), the incident feed grouped by day with the exact figures that fired each rule, and a sortable breakdown per tool and model with the confidence badge on every row. Light and dark follow the system or a setting; the palette is the system one, so it tracks the user's accent and contrast preferences.

Beyond the window

The richer views built this month — live sessions with the context gauge and prompt-cache countdown, the per-session drill-down with the subagent cost tree and "what grew the context", the period digest, and the what-if repricing — live in `packages/core` and are reachable from the terminal and from any agent over MCP (next section). Porting them into the app is the top item on the [roadmap](#).

Command line and integrations

Everything the app knows, and the views it does not draw yet, from the terminal — across every tool.

```
pnpm top          # htop for agents: live sessions, context vs window, tok/min, cache countdown,
flags
pnpm pr           # markdown table of agent usage on the current git branch, for a PR description
pnpm digest       # "your agent week" as markdown (--range=24h|7d|30d|all, --json)
pnpm statusline   # one line for a status bar (reads Claude Code's status-line JSON, or --
session=<id>)
pnpm mcp          # stdio MCP server over the same queries
```

`pnpm pr` attributes by working directory inside the repo since the branch diverged from the default branch, so it covers every tool; only Claude Code records the branch name itself, and that exact count is printed as a footnote rather than used as the filter.

Status line. Claude Code pipes a JSON object with `session_id` to the status-line command, so:

```
{ "statusLine": { "type": "command", "command": "pnpm --silent --dir /path/to/vole statusline" } }
```

prints `vole · ctx 412K/1M 41% · $3.20 · 1.2K tok/min · cache 2:40 · ! 1 warn` for the session you are in. Any other tool can call it with `--session=<id>`.

MCP. `pnpm --silent --dir /path/to/vole mcp` speaks newline-delimited JSON-RPC on stdio, with no dependencies, and exposes `vole_summary`, `vole_live_sessions`, `vole_session`, `vole_incidents`, `vole_breakdown`, `vole_whatif` and `vole_digest`. An agent can ask what its own session has cost, how full its context is, and whether Vole has flagged it — which makes cost-aware agents possible. Register it wherever you run agents:

```
# Claude Code
claude mcp add vole -- pnpm --silent --dir /path/to/vole mcp
```

```
# Codex: ~/.codex/config.toml
[mcp_servers.vole]
command = "pnpm"
args = ["--silent", "--dir", "/path/to/vole", "mcp"]
```

```
// OpenCode: opencode.json          // Cursor: ~/.cursor/mcp.json
{ "mcp": { "vole": { "type": "local", "command": ["pnpm", "--silent", "--dir", "/path/to/vole", "mcp"] } }
}
{ "mcpServers": { "vole": { "command": "pnpm", "args": ["--silent", "--dir", "/path/to/vole", "mcp"] } }
}
```

Nothing leaves the machine: the server reads the local database and answers on stdout.

Anomaly rules

Rules are pure functions over event arrays, unit-tested without a database. Each firing writes its own row so the dashboard can render an incident feed.

RULE	FIRES WHEN
burn_rate_spike	A 10-min window exceeds 3× the median window for that same tool+model (min 20K tokens)
loop_suspected	≥15 calls in 5 min at >3× the session's own rate, while output stays flat and cache reads climb
error_storm	>20% error ratio over 15 min, with ≥5 errors
rate_limit_pressure	Codex reports >80% of its quota consumed (the only tool that self-reports this)
context_pressure	A call carried ≥80% of the model's context window (≥95% critical); once per session per hour

context_pressure uses the window the tool reports (Codex) or the published one for first-party model ids in pricing.json . An OpenCode provider/model id resolves only for the anthropic provider, since a proxy provider may cap the window; anything unknown is skipped, never guessed.

Two design details that matter:

Baselines are leave-one-out. A window is compared against the median of *all other* windows. With a plain median, a lone spike drags up the very baseline it is measured against and hides inside it — especially with sparse data.

Loop detection needs two signals. High call volume alone also describes a productive burst. The distinguishing signature of a stuck agent is many calls where *output stays flat while cache reads climb* — it keeps re-reading the same context and producing nothing.

Cost model

Cost is **equivalent API value at list price** — what this usage *would* have cost through the API. On a subscription plan you are not billed per token, and the UI says so.

cost = (input·I + write5m·I·1.25 + write1h·I·2 + cacheRead·I·0.1 + output·0) / 1e6

Rates are data, not code: [packages/core/src/data/pricing.json](#) , versioned with effective_from . A per-installation ~/.vole/pricing.json (or \$VOLE_PRICING) merges over it, so new models can be added without a release:

MODEL	INPUT \$/MTOK	OUTPUT \$/MTOK
claude-fable-5-1 , claude-fable-5	10.00	50.00
claude-opus-5 , claude-opus-4-8	5.00	25.00
claude-sonnet-5	2.00	10.00

Cache multipliers on base input: read **0.1×**, 5-minute write **1.25×**, 1-hour write **2×**. A model may override the read rate with a flat `cache_read` \$/MTok (Fable 5.1 reads at 0.25). Dated snapshot ids (`claude-haiku-4-5-20251001`) price as their alias.

Rows stored before their model had a rate are priced retroactively the next time the collector starts, so adding a model to `~/.vole/pricing.json` reprices history, not just future calls.

Unknown or unpriceable models return `NULL`, never `0`. That includes `<synthetic>`, a real Claude Code entry type representing a message with no billed API call.

Verifying it works

```
pnpm test      # unit tests: rules, queries, bucketing, confidence invariants
pnpm verify    # reconciles every stored row against its own source record
```

`pnpm verify` is the important one. It:

- re-implements the cost formula **independently**, so a bug in the product's own pricing module cannot cancel itself out;
- compares **per record** rather than comparing totals — totals drift continuously while a session is live, so a total-vs-total check can neither prove nor disprove correctness;
- anchors Codex on the source's own **cumulative meter**: a stored total must equal the meter delta, and breakdown fields must be `NULL` (not `0`) where the source never split the meter;
- checks every `activity_only` row is `NULL` in all token and cost fields;
- reports the live duplication factor.

Expected output (one section per source):

```
[claude] reconciled per-message  7583
[claude] token mismatches        0
[claude] cost mismatches         0
[opencode] token mismatches      0
[codex] reconciled per-event     53
[codex] token mismatches         0
[codex] breakdown mismatches     0
[no-token] fields not NULL       0
```

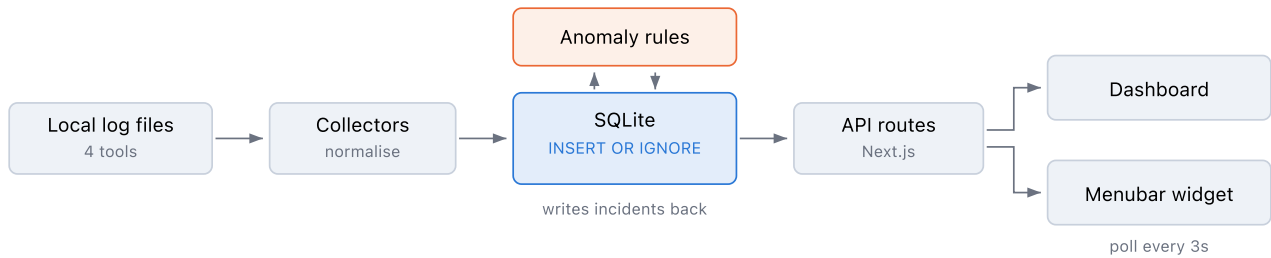
PASS — every stored row matches its source record.

If that final line ever changes, something is genuinely wrong.

Architecture

```
packages/core    types · SQLite schema + DAL · pricing · collectors · rules · queries · CLIs · MCP
apps/mac         SwiftUI menu-bar app + dashboard window, reading the same SQLite file read-only
```

`packages/core` is plain TypeScript with no UI imports — that is what lets the collector, the seeder, the verifier, the CLIs and the MCP server share one implementation. The app has no Node runtime: `DB.swift` ports the read models from `queries.ts` onto the SQLite3 C API that ships with macOS, so the two agree by construction on the same file.



Documentation

Full documentation lives in [docs/](#), and is also available as a single [combined PDF](#) (46 pages, with bookmarks) for offline reading or sharing:

GUIDE	FOR
ARCHITECTURE.md	Module map, schema DDL, idempotency contract, how the app reads the store
DATA-SOURCES.md	What every tool writes to disk, with real samples and traps
DEVELOPMENT.md	Setup, commands, debugging recipes, testing philosophy
EXTENDING.md	Add a tool, a rule, a model rate, or a panel in the app
DECISIONS.md	How it was built, and the five bugs that shaped it
ROADMAP.md	Ranked next steps, plus good first issues

Regenerate the PDF after editing any markdown with `pnpm docs:pdf`.

Known limitations

- **Codex and Cursor data may be stale.** On the development machine, Codex data was ~9 months old and Cursor ~4 months. Both parsers are real and correct, but they contribute history rather than live activity unless you actively use those tools.
- **is_error covers API errors only** (`isApiErrorMessage`). Failed *tool* results inside a turn are not yet correlated back to the assistant message that issued them, so `error_storm` under-counts on live data.
- **Antigravity timing is approximate** — file mtimes, not real timestamps, so its events cluster rather than spread across a session.
- **The app is ad-hoc signed.** `bundle.sh` produces a runnable `Vole.app`, but distribution outside your own machine needs an Apple Developer ID and notarisation.
- **Live sessions, session drill-down, digest and what-if are not drawn in the app yet.** They are complete in core and available through `pnpm top`, `pnpm digest` and the MCP server.
- Cursor and Antigravity coverage is **deliberately shallow** because the data genuinely is not there. That is documented rather than disguised.
- **Databases collected before tool names, agent ids and context windows existed** keep those columns NULL on old rows (the upsert only rewrites a row when its tokens grow). The database is disposable: `rm ~/.vole/vole.db*` and `pnpm collect:once` rebuilds it from the logs.
- **Context windows are known only for first-party model ids** listed in `pricing.json` (`context_windows`) or reported by the tool itself (Codex). Local and proxied models show a context size but no window, and never trip `context_pressure`.

Licence

MIT — see [LICENSE](#).

How this was built

An engineering record: the method, the decisions, and — most usefully — the bugs that were caught and how. If you are picking this project up, this is the file that explains *why* the code looks the way it does.

The method: investigate before designing

The original brief specified four data sources and assumed each one worked a particular way. The first action was **not** to write a parser. It was to look at what is actually on disk.

That investigation changed the plan substantially:

BRIEF ASSUMED	REALITY FOUND
Codex uses <code>~/codex/projects/</code> JSONL like Claude Code	It uses <code>~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl</code> , and emits <i>cumulative</i> counters
Cursor persistence "less standardised, investigate first"	Cursor stores no token data at all locally
Antigravity "token counts not exposed exactly"	Correct — and worse: conversation payloads are encrypted
Two confidence levels (<code>exact</code> / <code>estimated</code>) suffice	Cursor fits neither; a third level was required

Takeaway for anyone extending this: an hour spent with `jq`, `sqlite3` and `xxd` before writing code saved several wrong parsers. The investigation method is documented in [DATA-SOURCES.md](#).

The five bugs that shaped the architecture

Each of these was caught by verification rather than by reading code. They are listed first because they explain most of the non-obvious decisions.

1. Claude Code duplicates every usage record (~2.1x)

Found: counting raw usage rows (861) against unique `message.id` values (408) during the initial investigation.

Impact if missed: every token count, every cost, and every anomaly baseline roughly doubled. The product would have looked like it worked while being wrong everywhere.

Fix: `event_key = claude_code:<message.id>` with a `UNIQUE` constraint and `INSERT OR IGNORE`.

Why this shape: dedup could have been a `SELECT DISTINCT` at query time, but making it a database constraint means the *polling loop* is also idempotent — the collector never has to ask "have I seen this?". One decision solved two problems.

Guard: `pnpm verify` prints the live duplication factor on every run.

2. Codex double-counting, twice over

Found: a Codex total of 120M tokens looked implausible for five small sessions, so it was checked against each session's own final cumulative counter.

Two distinct traps sat here:

- Summing `total_token_usage` (cumulative) grows quadratically with turn count.

- Summing `last_token_usage` (per-turn) *also* over-counts, because Codex sometimes emits the same `token_count` event twice.

Fix: derive usage from the **delta of the cumulative counter**. A duplicate emission produces a delta of zero and is skipped.

Verification: per-session sums reconstruct each session's final cumulative counter *exactly* (12,867,036 / 1,333,250 / 15,989,943 / 89,931,567 / 80,975). The 120M turned out to be real — one session ran 1,513 turns, and cached context is re-read every turn.

Lesson: "this number looks too big" is a hypothesis, not a conclusion. It was worth checking, and the check proved the code right rather than wrong.

3. Float division silently destroyed time bucketing

Found: API output showed bucket values like `1786160794400.9998` — not day boundaries.

Cause: a bound numeric parameter makes SQLite use floating-point division, so `(ts / n) * n` returns `ts` unchanged. Every event became its own bucket: the chart would have drawn 1,579 bars instead of 17.

Fix: `CAST(ts / ? AS INTEGER) * ?`, plus a regression test asserting that events seconds apart collapse into one day bucket.

Lesson: the bug was invisible in the UI at a glance — the chart *rendered*, it was just wrong. Checking the API payload rather than the picture is what caught it.

4. Anomaly baselines masked the anomalies

Found: a unit test that *should* have passed did not. With windows of `[10, 30]`, the median is 20, so 30 calls failed a `> 3×` test it should obviously have passed.

Cause: the outlier was included in the baseline it was being measured against.

Fix: `medianExcluding()` — leave-one-out baselines. A window is compared against the median of all *other* windows.

Lesson: the test caught a real design flaw, not a coding slip. It mattered because real data is sparse, which is exactly when a single window dominates the median.

5. Seed data contaminated live baselines

Found: after seeding, burn-spike incidents could not be attributed to a source.

Cause: detection ran over live and seeded rows *pooled together*. Keeping them in separate rows is not enough if the **analysis** mixes them — 30 days of synthetic history would redefine what "normal" means for real usage, and a genuine spike could be masked by demo data.

Fix: `detectBySource()` partitions events by `source` before running rules, and folds the source into `anomaly_key`.

Lesson: "don't contaminate real data" is a statement about the statistics, not just the storage.

Design decisions

Confidence is a three-value system

The brief specified `exact | estimated`. Cursor has real sessions and models but **zero tokens**. Labelling that "estimated" would mean fabricating numbers — precisely what the confidence system exists to prevent.

`activity_only` rows store `NULL` (never `0`) and are excluded from every token and cost aggregate while still counting as calls.

Rejected alternative: estimating Cursor tokens from lines of code. It would have produced a plausible-looking number with an "estimated" badge, and a demo reviewer could not have told the difference between that and Antigravity's defensible turn-count proxy. Fabrication that looks like measurement is worse than an honest gap.

Exact tokens ≠ known cost

Codex gives exact tokens, but no published per-token rate is loaded, so `cost_usd` is `NULL` and renders `–`. Two separate ideas, two separate representations. Rendering `$0` would have been a lie of a different kind.

Cost is "equivalent API value", labelled

On a subscription plan you are not billed per token. Presenting a dollar figure as a bill would be misleading, and omitting it entirely loses the number people react to. The compromise is to compute list-price equivalent and **say so on the tile**.

It is computed *exactly*, not approximated: Claude Code reports cache creation already split by TTL, so each component gets its own multiplier (read 0.1x, 5m write 1.25x, 1h write 2x).

Rules are pure functions

`(events, now) => Anomaly[]`, with no database access. This makes them unit-testable against fixture arrays, which is what allowed the leave-one-out flaw to surface in a test rather than in production data.

Loop detection needs two signals

Call frequency alone does not distinguish a stuck agent from a productive burst. The rule requires high call volume **and** the stuck signature: output near-flat while cache reads climb.

This is validated by a negative test — 45 rapid calls *with* real output must **not** fire.

Idempotency as a database constraint

Every collector produces a stable natural key; the schema enforces uniqueness; inserts ignore conflicts. As a result:

- re-scanning a file is free and safe,
- `collector_state` offsets are a pure optimisation, not a correctness requirement,
- deleting `collector_state` triggers a harmless full re-parse.

Verification compares per message, not by totals

Totals drift continuously while a Claude Code session is live — the logs grow underneath you. A total-vs-total check can therefore neither prove nor disprove correctness.

`pnpm verify` reconciles **each stored row against its own source record**, and deliberately re-implements the cost formula rather than importing `pricing.ts`, so a bug there cannot cancel itself out.

This distinction was not academic: an early total comparison showed `$40.12` expected vs `$39.83` stored. Both numbers were correct; the gap was drift. Per-message reconciliation resolved it to 400/400 with zero mismatches.

Interface decisions

Superseded (September 2026). The web dashboard and the Tauri widget described in the next few sections were removed: the product is the native macOS app in `apps/mac` plus the collector. The notes stay because the reasoning — chart animation versus polling, the palette relief rule, one UI over one data path — carried straight into the SwiftUI app.

Recharts, not Tremor

The brief specified Tremor + React 19. Tremor 3.18 declares `react: ^18.0.0` and does not support React 19. Rather than force a peer-dependency override in a time-boxed build, the project took the brief's own stated fallback — Recharts, which supports React 19 explicitly — and hand-built the stat tiles.

The signature: incidents pinned to the timeline

The first UI kept the usage chart and the incident list separate, which made the reader do the correlation themselves: see a spike, hunt the list for the cause.

Pinning each incident onto the exact bucket where its rule fired collapses that into one glance, and the highlight is bidirectional. This is information design, not decoration — it is the one element the tool is remembered by.

Chart animation is disabled

Recharts animates entry over 1.5s; the dashboard polls every 3s. Left on, the chart re-animates from zero on every refresh — flickering roughly half the time. A live monitor should look stable.

This also explains why two early screenshots showed an "empty" chart: they caught it mid-animation. The DOM inspection that diagnosed it (`document.querySelectorAll('.recharts-bar-rectangle path')` returned 70 elements with correct fills) is the debugging move worth remembering.

Palette was re-validated, not assumed

The categorical palette was validated against a *warm* dark surface. This design uses cool graphite, so the earlier results no longer applied and the palette was re-run against the actual surfaces.

Dark passes every check. Light passes separation checks but two slots fall below 3:1 contrast, so the **relief rule** applies — the dashboard ships a labelled legend *and* a full breakdown table, so no value is ever carried by colour alone.

One UI, two windows

The Tauri widget loads the dashboard's own `/widget` route rather than reimplementing a second interface. One component set, one polling path, one design system. The Rust side contributes only the desktop shell.

The live menubar title is pushed *from* the page *into* Rust via a single command, which avoids giving the Rust binary its own HTTP client and compile cost.

What was deliberately not built

Recorded so nobody re-litigates them without new information.

NOT BUILT	WHY
Cursor token estimates	Would be fabricated. No local source exists.
Antigravity payload decryption	Encrypted; attempting it is out of scope and fragile.
WebSocket live updates	Polling is simpler and self-heals. The brief said add it only if polling was solid and time allowed.
Schema migrations	The database is disposable — it rebuilds from source logs in seconds.
Cloud sync / multi-machine	The project's premise is local-only.
Web-UI scraping for any tool	Explicitly out of scope, and fragile by nature.

Honest limitations

These are real and currently unaddressed. See [ROADMAP.md](#) for what to do about them.

- `is_error` covers API errors only. Failed `tool` results inside a turn are not correlated back to the assistant message that issued them, so `error_storm` under-counts on live data. It fires reliably on seeded data only.
 - Antigravity timing is approximate — file mtimes, not real timestamps.
 - Codex and Cursor data may be stale depending on your usage; their parsers are correct but may contribute only history.
 - Production build untested. Only `next dev` has been exercised.
 - The app is ad-hoc signed and not packaged for distribution.
-

The habit worth copying

Nearly every fix above came from the same move: **check the claim against the source rather than against the code.**

- Raw log counts, not the parser's opinion.
- API payloads, not the rendered chart.
- The DOM, not the screenshot.
- An independently re-implemented formula, not the one under test.

The single most valuable artefact in this repo is `pnpm verify`, because it makes that habit automatic.

Architecture

Reference for how Vole is put together, and why the non-obvious decisions were made. For what the product does and how to run it, see the [README](#).

Module map

packages/core/src/	
types.ts	Shared types. Confidence levels, UsageEvent, Anomaly.
schema.ts	SQLite DDL as an inlined string.
db.ts	Connection, migrations, idempotent inserts, repricing, seed purge.
pricing.ts	Versioned rate table, cache multipliers, context windows, computeCost().
paths.ts	Source locations, with env overrides for tests.
queries.ts	Read models: summary, timeseries, breakdown, anomalies, live sessions, session detail, cache re-warm, what-if, digest.
digest.ts	The digest as markdown.
util/jsonl.ts	Incremental byte-offset line reader.
util/format.ts	Compact numbers, money, countdowns – for the CLIs.
collectors/	
claude-code.ts	Primary source. Exact. Deduped on message.id. Recursive (subagents).
codex.ts	Exact tokens via cumulative deltas; tool names; self-reported window.
cursor.ts	Activity only – no tokens exist locally.
antigravity.ts	Activity only; no local token data.
opencode.ts	Exact per-message tokens + cost from opencode.db; tools; child sessions.
grok.ts	Exact per-turn tokens; tool executions; failed inferences as error rows.
devin.ts	Activity only; ACP payloads carry no token data.
index.ts	Registry; isolates per-collector failures.
detect/	
burn-rate.ts	Token-burn rate spike.
loop.ts	Runaway loop.
error-storm.ts	Retry/failure ratio spike.
rate-limit.ts	Quota pressure (Codex only).
context-pressure.ts	Context carried vs the model window.
util.ts	Bucketing, leave-one-out median, confidence propagation.
index.ts	detectAll() and detectBySource().
cli/	
collect.ts	The monitor. One-shot or polling; desktop notifications.
seed.ts	Demo data generator and purge.
verify.ts	Per-message reconciliation against raw logs.
top.ts	Live sessions in the terminal.
pr.ts	Usage on the current branch, as markdown.
digest.ts	Period digest, markdown or JSON.
statusline.ts	One line for a status bar.
mcp.ts	stdio MCP server over the queries.
apps/mac/Sources/Vole/	
DB.swift	Read-only SQLite3 + the read models ported from queries.ts.
Store.swift	@Observable state, re-reads on the chosen interval.
Theme.swift	Palette, formatters, labels, tool logos.
MenuPanel.swift	The menu-bar panel.
DashboardView.swift	The window: KPI row, timeline (Swift Charts), incidents, breakdown, settings.
VoleApp.swift	MenuBarExtra + Window scenes, activation policy.

`packages/core` imports no UI framework. That constraint is what lets the collector, the seeder, the verifier, the CLIs and the MCP server all share one implementation of parsing, pricing and detection. The app has no Node

runtime at all: it reads the same file through the SQLite3 library that ships with macOS.

Database schema

Written to `~/vole/vole.db` (override with `VOLE_DB`). WAL mode.

usage_events

One row per model call, normalised across all tools.

```
CREATE TABLE usage_events (
  id                INTEGER PRIMARY KEY AUTOINCREMENT,
  event_key         TEXT    NOT NULL UNIQUE,    -- dedup key
  tool              TEXT    NOT NULL,          --
  claude_code|codex|cursor|antigravity|opencode|grok|devin
  model             TEXT,
  session_id        TEXT,
  project           TEXT,                      -- cwd / workspace
  git_branch        TEXT,
  ts                INTEGER NOT NULL,          -- epoch ms UTC
  input_tokens       INTEGER,
  output_tokens      INTEGER,
  cache_write_5m_tokens INTEGER,
  cache_write_1h_tokens INTEGER,
  cache_read_tokens  INTEGER,
  reasoning_tokens   INTEGER,
  total_tokens       INTEGER,
  cost_usd           REAL,                    -- NULL when rate unknown
  confidence         TEXT    NOT NULL,        -- exact|activity_only
  is_error           INTEGER NOT NULL DEFAULT 0,
  stop_reason        TEXT,
  source             TEXT    NOT NULL DEFAULT 'live', -- live|seed
  raw_ref            TEXT                      -- "<file>#<line>" for auditability
);
```

Nullable token columns are deliberate. A tool that records no tokens stores `NULL`, not `0`, so an absence can never be mistaken for a measurement of zero.

anomalies

```
CREATE TABLE anomalies (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  anomaly_key TEXT NOT NULL UNIQUE,  
  rule        TEXT NOT NULL,  
  severity    TEXT NOT NULL,          -- info|warn|critical  
  tool        TEXT NOT NULL,  
  session_id  TEXT,  
  model       TEXT,  
  window_start INTEGER NOT NULL,  
  window_end  INTEGER NOT NULL,  
  title       TEXT NOT NULL,  
  detail      TEXT NOT NULL,          -- human explanation carrying real numbers  
  observed    REAL NOT NULL,  
  baseline    REAL,  
  threshold   REAL,  
  confidence  TEXT NOT NULL,          -- worst confidence among contributing events  
  source      TEXT NOT NULL DEFAULT 'live',  
  detected_at INTEGER NOT NULL  
);
```

collector_state

Byte offsets so polling only reads newly appended bytes.

```
CREATE TABLE collector_state (  
  source_path TEXT PRIMARY KEY,  
  tool        TEXT NOT NULL,  
  last_offset  INTEGER NOT NULL DEFAULT 0,  
  last_mtime  INTEGER,  
  last_scanned_at INTEGER  
);
```

Idempotency: the event_key contract

Every collector produces a stable, natural key under a **UNIQUE** constraint, so re-scanning a file can never double-count.

TOOL	KEY	WHY
Claude Code	claude_code:<message.id>	Kills the ~2.4× transcript duplication
Codex	codex:<session_id>:<line_index>	Stable across full-file re-reads
Cursor	cursor:<requestId>	One row per model call, not per code hash
Antigravity	antigravity:<conversation_id>:<mtime>	New row only when the conversation changes
OpenCode	opencode:<message.id>	One row per assistant turn
Grok	grok:<session>:<timestamp>	One row per inference; log is re-read in full
Devin	devin:<session>:<turnId>	Activity only; one row per agent turn

The insert is `INSERT ... ON CONFLICT(event_key) DO UPDATE ... WHERE excluded.total_tokens > total_tokens`. For every collector but Claude Code the conflict clause is dead weight (their keys are immutable or already delta-based). Claude Code needs it: it writes each message to the transcript several times while streaming, the first copy with `output_tokens: 0`, so a plain ignore-on-conflict would freeze the row at zero output. The upgrade only ever raises a stored row's token count, so re-reading identical data is still a no-op.

This is the property that makes the polling loop safe. The collector never has to reason about "have I seen this before?" — the database answers it.

Why Codex re-reads whole files

Codex usage is derived from the *delta* of a cumulative counter, so it needs a known starting point. Resuming from a byte offset mid-file would lose the running total. Rollout files are few and small, and `INSERT OR IGNORE` makes the re-read free, so the collector reads them in full and recomputes deltas deterministically.

The delta approach also self-corrects for Codex emitting the same `token_count` event twice: a duplicate contributes a delta of zero and is skipped.

Detection design

All rules are pure: `(events, now) => Anomaly[]`. No database access, so they are unit-tested against fixture arrays.

Leave-one-out baselines

A window is compared against the median of *all other* windows, never a median that includes itself.

With a plain median and sparse data, a single large window drags the baseline up far enough to mask itself. Example: windows of `[10, 30]` have a median of 20, so 30 calls fails a `> 3×` test that it should obviously pass. Excluding the candidate gives a baseline of 10, and the rule fires correctly.

Two-signal loop detection

Call frequency alone does not distinguish a stuck agent from a productive burst. The rule requires **both** high call volume *and* the stuck signature: average output near-flat while cache reads climb — an agent re-reading the same context and producing nothing.

Source partitioning

`detectBySource()` runs the rules separately for each `source` value and folds the source into `anomaly_key`.

Keeping seeded and live rows in separate *rows* is not enough if the *analysis* pools them: a 30-day synthetic history would redefine what "normal" means for real usage, and a genuine spike could be masked by demo data. Partitioning is what makes demo data genuinely removable.

Anomaly key stability

Keys bucket by rule + tool + session + time window, and never include `now`. A sustained incident therefore produces one row rather than one row per poll, and re-running detection is idempotent.

Confidence propagation

`worstConfidence()` takes the weakest confidence among the events contributing to an anomaly. An incident is only as trustworthy as its least trustworthy input.

Token maths filters `activity_only` rows out entirely rather than coalescing them to zero — counting a call with unknown size as zero would silently drag down per-call averages.

The read models enforce the same rule:

```
-- excluded from token sums, still counted as calls
COALESCE(SUM(CASE WHEN confidence != 'activity_only' THEN total_tokens END), 0)
```

How the app reads the store

`DB.swift` opens `~/vole/vole.db` (or `$VOLE_DB`) read-only through the system `SQLite3` module and ports the read models from `queries.ts` one to one — same `confidence != 'activity_only'` guard on every token aggregate, same `source = 'live'` filter, same bucketing. `Store.swift` runs them on a timer (5s "live", or longer from Settings) on the main actor; the whole refresh is a few milliseconds on a 10 MB database, so nothing is moved off-main until it shows up as a hitch.

The app distinguishes *quiet* from *collector down* with `collector_state.last_scanned_at`: the collector stamps every source it touches on every pass, so `MAX(last_scanned_at)` is its heartbeat.

Bucketing needs an explicit `CAST` in both implementations. A bound numeric parameter makes SQLite use floating-point division, so `(ts / n) * n` returns `ts` unchanged and every event becomes its own bucket. `CAST(ts / ? AS INTEGER) * ?` is what actually aligns buckets to day/hour edges.

The UI is Apple-native on purpose: system semantic colours and the standard controls, one brand colour per tool that is never reused for status, severity always as a coloured glyph *with a word*, and Liquid Glass only where the system supplies it (the menu-bar panel, the toolbar) — never nested. See [apps/mac/README.md](https://github.com/yourusername/apps/blob/main/mac/README.md) for building and packaging.

Data sources

Reference for what each AI coding tool writes to disk, what it means, and where it lies to you.

This is the knowledge that took the most work to establish, and it is the part most likely to break as tools ship new versions. If you are adding a tool or debugging a wrong number, start here.

Rule of the project: never invent a number. If a tool does not record something, the field stays `NULL` and the UI says so. See [confidence levels](#) at the end.

Claude Code — exact

Location: `~/.claude/projects/<url-encoded-project-path>/<session-uuid>.jsonl`

The directory name is the project's absolute path with `/` replaced by `-`, e.g. `/Users/you/projects/app` becomes `-Users-you-projects-app`. One JSONL file per session.

Each line is a JSON object. Many `type` values appear (`queue-operation`, `user`, `assistant`, attachments, hook results). **Only `type: "assistant"` entries with `message.usage` carry token data.**

A real entry

```
{
  "type": "assistant",
  "timestamp": "2026-07-26T03:20:02.352Z",
  "sessionId": "3a5594ab-855c-470b-bf02-89f1758c2e04",
  "requestId": "req_011CdPwwPinvfWd5hiApdpmS",
  "cwd": "/Users/you/projects/app",
  "gitBranch": "HEAD",
  "version": "2.1.220",
  "message": {
    "id": "msg_011CdPwwR3fUBkqb4r9ExxLG",    // ← the dedup key
    "model": "claude-sonnet-5",
    "stop_reason": "end_turn",
    "usage": {
      "input_tokens": 2,
      "cache_creation_input_tokens": 280,
      "cache_read_input_tokens": 368480,
      "output_tokens": 32,
      "cache_creation": {                      // ← split by TTL; this is what makes cost exact
        "ephemeral_5m_input_tokens": 0,
        "ephemeral_1h_input_tokens": 280
      },
      "output_tokens_details": { "thinking_tokens": 0 },
      "server_tool_use": { "web_search_requests": 0, "web_fetch_requests": 0 },
      "service_tier": "standard",
      "iterations": [ /* per-iteration breakdown; usually one entry */ ]
    }
  }
}
```

Field meanings

FIELD	MEANING
message.id	Stable API response id. The dedup key.
input_tokens	Fresh (uncached) input. Often tiny — most input is cached.
cache_read_input_tokens	Input served from cache, billed at 0.1×. Usually the largest number.
cache_creation.ephemeral_5m_input_tokens	Cache written with 5-minute TTL, billed at 1.25×.
cache_creation.ephemeral_1h_input_tokens	Cache written with 1-hour TTL, billed at 2×.
cache_creation_input_tokens	Total of both TTLs. Present on older entries that lack the split.
output_tokens	Generated tokens.
output_tokens_details.thinking_tokens	Reasoning tokens (subset of output).
stop_reason	tool_use dominates in agentic sessions; end_turn ends a turn.
isApiErrorMessage	Top-level (not under message). Marks an API error.

⚠ The duplication trap

Claude Code writes the same API response to the transcript more than once.

Measured on real logs: 861 raw usage rows → 408 unique message.id . Duplicates carry byte-identical usage, with different uuid and timestamps milliseconds apart.

Summing rows naively inflates every number by roughly 2.1×.

```
# See it yourself
cat ~/.claude/projects/**/*.jsonl | jq -r 'select(.message.usage!=null) | "x"' | wc -l # raw
cat ~/.claude/projects/**/*.jsonl | jq -r 'select(.message.usage!=null) | .message.id' \
| sort -u | wc -l # unique
```

The collector keys events on claude_code:<message.id> under a UNIQUE constraint. pnpm verify reports the live duplication factor so a regression here is loud.

Other gotchas

- model can be "<synthetic>" — a real entry type for messages with no billed API call. It must produce no cost, not \$0 computed from a missing rate.
- The last line may be partial while a session is live. The reader deliberately leaves an incomplete trailing line unconsumed and re-reads it next poll.
- Older entries lack cache_creation and only have the flat total. The collector attributes those to the 5-minute bucket, since that is the default TTL.

Codex CLI — exact tokens, unknown cost

Location: ~/.codex/sessions/YYYY/MM/DD/rollout-<timestamp>-<uuid>.jsonl

Not ~/.codex/projects/ . Codex also keeps SQLite databases in ~/.codex/ (logs_2.sqlite , state_5.sqlite) — state_5.sqlite has a threads table with a tokens_used column that is useful as a cross-check, but the rollout JSONL is the authoritative per-turn source.

Line types

TYPE	CONTAINS
session_meta	Session id, cwd. <code>model</code> is <code>null</code> here — do not read it.
turn_context	<code>model</code> , <code>effort</code> , <code>sandbox policy</code> . This is where the model name lives.
event_msg	Wraps <code>payload.type</code> values including <code>token_count</code> , <code>agent_message</code> .
response_item	Model output items.

A real `token_count` payload

```
{
  "type": "token_count",
  "info": {
    "total_token_usage": {           // ← CUMULATIVE for the session
      "input_tokens": 79595,
      "cached_input_tokens": 58240,
      "output_tokens": 1380,
      "reasoning_output_tokens": 448,
      "total_tokens": 80975
    },
    "last_token_usage": {           // ← this turn only
      "input_tokens": 10223,
      "cached_input_tokens": 9984,
      "output_tokens": 145,
      "reasoning_output_tokens": 64,
      "total_tokens": 10368
    },
    "model_context_window": 258400
  },
  "rate_limits": {
    "primary": { "used_percent": 1.0, "window_minutes": 300, "resets_at": 1764406328 },
    "secondary": { "used_percent": 0.0, "window_minutes": 10080, "resets_at": 1764993128 },
    "credits": { "has_credits": false, "unlimited": false, "balance": "0" }
  }
}
```

⚠ Two traps

1. Do not sum `total_token_usage`. It is cumulative. Summing it across events produces a number that grows quadratically with turn count.
2. Do not blindly sum `last_token_usage` either. Codex sometimes emits the *same* `token_count` event twice — two identical events appear at session start in real logs.

The collector uses the **delta of the cumulative counter**, which is immune to both:

```
usage = total_token_usage(now) - total_token_usage(previous)
```

A duplicate emission produces a delta of zero and is skipped. Correctness of this approach is provable: per-session sums reconstruct each session's final cumulative counter exactly.

Because deltas need a known starting point, Codex rollouts are **re-read in full** each poll rather than resumed from a byte offset. The files are few and small, and `INSERT OR IGNORE` makes the re-read free.

Cached input is nested, not separate

`input_tokens` **includes** `cached_input_tokens`. The collector separates them so the cache maths holds:

```
const freshInput = Math.max(0, input - cached);
```

Cost is unavailable

Tokens are exact, but no published per-token rate for `gpt-5.1-codex-max` is loaded, so `cost_usd` stays `NULL` and renders as `-`. **Exact tokens do not imply known cost.** To add a rate, see [EXTENDING.md](#) → [Adding a model rate](#).

Bonus signal

Codex is the only tool that reports **its own remaining quota** (`rate_limits.primary.used_percent`). That powers the `rate_limit_pressure` rule, which has no equivalent for the other tools.

Cursor — no token data exists locally

Location: `~/.cursor/ai-tracking/ai-code-tracking.db` (SQLite)

What is actually in it

```
CREATE TABLE ai_code_hashes (
  hash TEXT PRIMARY KEY,
  source TEXT NOT NULL,      -- e.g. 'composer'
  fileExtension TEXT,
  fileName TEXT,
  requestId TEXT,
  conversationId TEXT,
  timestamp INTEGER,
  model TEXT,                -- e.g. 'composer-2-fast'
  createdAt INTEGER NOT NULL
);
-- also: scored_commits, conversation_summaries, tracked_file_content, tracking_state
```

There are no token columns anywhere in this database. It exists to power Cursor's "% AI-written code" feature by hashing generated code, not to account for usage.

`~/Library/Application Support/Cursor/User/globalStorage/state.vscdb` was also checked: it is large (400MB+) but its `ItemTable` holds no usage or token keys.

Verify for yourself:

```
sqlite3 ~/.cursor/ai-tracking/ai-code-tracking.db ".schema" | grep -i token # → nothing
```

What the collector does

Emits **one row per** `requestId` (a model call), not per code hash — 715 hashes on the development machine corresponded to only 8 actual requests. All token fields are `NULL` and confidence is `activity_only`.

Estimating tokens from lines of code was considered and rejected. It would be a fabricated number wearing an "estimated" badge. Cursor's real accounting lives server-side behind its account, and this project does not automate web UIs.

The DB is opened **read-only** because Cursor may be running and holding the file.

Antigravity — activity only

Locations:

- `~/.gemini/antigravity-ide/conversations/<uuid>.pb` — **encrypted**
- `~/.gemini/antigravity-ide/brain/<uuid>/` — plaintext artifacts

Why there is no token count

Two hard constraints:

1. Antigravity is a **flat-rate subscription** product and exposes no client-readable per-token usage anywhere on disk.
2. Conversation payloads are **encrypted**. Byte inspection shows high-entropy data with no protobuf wire format:

```
$ head -c 32 ~/.gemini/antigravity-ide/conversations/<uuid>.pb | xxd
00000000: eb74 86cc 6a47 0091 e908 6e6d 6f90 fef3
```

A protobuf stream starts with a field tag byte; `0xeb` is not a plausible one, and `file(1)` reports generic `data`.

What is readable

`brain/<conversation-id>/` contains plaintext artifacts the agent wrote — implementation plans, task files, and numbered `.resolved.N` revisions. Their **count is a turn-count proxy**, nothing more.

```
brain/937423ff-.../
  implementation_plan.md
  implementation_plan.md.metadata.json
  implementation_plan.md.resolved.0
  implementation_plan.md.resolved.1
  ...
```

That proxy is **not** turned into a token number. Multiplying it by a guessed constant was considered and rejected — it would be a fabricated figure wearing a badge. Vole emits one `activity_only` row per conversation, with `NULL` token fields, so Antigravity shows up as real activity in the call count and nowhere in the token or cost totals.

Timing is approximate — file mtimes, not real timestamps — so Antigravity events cluster rather than spreading across a session.

Confidence levels

Two, by policy. A token count is measured or it is absent; there is no derived tier.

LEVEL	MEANING	TOKEN FIELDS	IN AGGREGATES?
<code>exact</code>	Read verbatim from the tool's logs (a plain sum, or a delta of the tool's own counter)	Real numbers	Yes
<code>activity_only</code>	Call happened; tool records no tokens (Cursor, Antigravity)	NULL	No (still counted as a call)

`activity_only` covers the tools that record real sessions and models but no token counts — Cursor and Antigravity. Deriving a number for them from lines of code or turn counts was considered and rejected: it would be a fabricated figure, exactly what this project refuses to produce.

NULL rather than `0` matters: an absence must never be mistaken for a measurement of zero.

Investigating a new tool

The method that produced everything above:

```
# 1. Find where it stores state
ls -la ~/.<tool> ~/.config/<tool> "~/Library/Application Support/<Tool>" 2>/dev/null

# 2. Find the biggest / most recent files – usage logs are usually both
du -sh ~/.<tool>/* | sort -h | tail

# 3. If JSONL: what line types exist, and do any mention tokens?
cat <file>.jsonl | jq -r '.type // "none"' | sort | uniq -c
grep -c token <file>.jsonl

# 4. If SQLite: does the schema even have token columns?
sqlite3 <file>.db ".schema" | grep -i token

# 5. If binary: is it parseable or encrypted?
file <file>; head -c 64 <file> | xxd
```

Then ask the one question that decides the confidence level:

1. Are there **real token counts** in the source? → `exact` (sum the real fields, or delta a cumulative counter — never a proxy times a constant)
2. Calls are recorded but no token data exists? → `activity_only` , **NULL** tokens

If the answer is "I could guess", the answer is `activity_only` . Guessing is the one thing this project does not do.

Next: [EXTENDING.md](#) → [Adding a collector](#).

Development guide

Everything needed to get productive in this codebase, including how to debug it when a number looks wrong.

Prerequisites

REQUIREMENT	VERSION	NEEDED FOR
Node	≥ 22	The collector, CLIs, tests
pnpm	≥ 9	Workspace management
Xcode 26	with the macOS 26 SDK	The app (<code>swift build</code>); its CLT also compiles <code>better-sqlite3</code>

Built and tested on **macOS 26.5 (arm64)**. The collectors read macOS/Linux paths; Windows would need `paths.ts` updated.

```
git clone https://github.com/launchsafe/vole
cd vole
pnpm install
```

`better-sqlite3` compiles a native binding during install. If that fails, you are missing Xcode CLT (`xcode-select --install`).

First run

```
pnpm collect:once    # parse local logs, print a summary
pnpm app             # the menu-bar app (swift run)
```

If `collect:once` prints all zeros, you have no local agent logs — run `pnpm seed` for demo data.

Commands

COMMAND	WHAT IT DOES
<code>pnpm collect:once</code>	One parse pass over all sources, with a verbose summary
<code>pnpm collect</code>	Continuous polling every 5s — this is the monitor
<code>pnpm app</code>	The macOS app via <code>swift run</code> (no bundle)
<code>pnpm app:bundle</code>	<code>build/Vole.app</code> , release build, ad-hoc signed, opened
<code>pnpm test</code>	Unit tests (rules, queries, bucketing, invariants)
<code>pnpm typecheck</code>	<code>tsc --noEmit</code> in every package (<code>tsx</code> strips types without checking them)
<code>pnpm verify</code>	Per-message reconciliation against raw logs
<code>pnpm seed</code>	Generate 30 days of demo data
<code>pnpm seed:purge</code>	Remove every seeded row
<code>pnpm top</code>	Terminal view of live sessions (<code>--once</code> for a single frame, <code>--since=<min></code>)
<code>pnpm pr</code>	Agent usage on the current git branch as markdown (<code>--since=<days></code> , <code>--json</code>)
<code>pnpm digest</code>	Period digest as markdown (<code>--range=</code> , <code>--json</code>)
<code>pnpm statusline</code>	One status-bar line for a session (stdin JSON with <code>session_id</code> , or <code>--session=</code>)
<code>pnpm mcp</code>	stdio MCP server; run with <code>pnpm --silent</code> so only protocol JSON reaches stdout

Collector flags:

```
pnpm --filter @vole/core collect -- --once           # single pass
pnpm --filter @vole/core collect -- --once --verbose # + summary tables
pnpm --filter @vole/core collect -- --interval=30    # poll every 30s
pnpm --filter @vole/core collect -- --no-notify      # no desktop notifications for new incidents
```

App:

```
swift run --package-path apps/mac Vole --dump      # headless: print the parsed summary and exit
swift run --package-path apps/mac Vole --range=7d  # open with a range preset
VERSION=0.2.0 apps/mac/bundle.sh                  # package build/Vole.app
```

Project layout

packages/core/	Plain TypeScript. No UI imports.
src/collectors/	One file per tool. Returns normalised rows; never touches SQL.
src/detect/	One file per rule. Pure functions, unit-tested without a DB.
src/cli/	collect (the monitor), seed, verify, top, pr, digest, statusline, mcp.
src/db.ts	Connection, migrations, idempotent inserts, repricing.
src/queries.ts	Read models – shared by the CLIs and the MCP server, ported to Swift.
src/pricing.ts	Rate table, context windows, computeCost().
apps/mac/	SwiftUI, Swift Package, no dependencies.
Sources/Vole/DB.swift	Read-only SQLite + read models ported from queries.ts.
Sources/Vole/Store.swift	Polling state.
Sources/Vole/*View.swift	Menu-bar panel and dashboard window.

The important constraint: `packages/core` imports no UI framework, and the app has no Node runtime. Anything the app shows must be a query that can be expressed in `queries.ts` and ported to `DB.swift`; if a query is only useful in one place, it still starts in core so `pnpm test` can cover it.

Layering rules

collectors	→ return UsageEvent[]	(never write SQL)
db	→ writes rows	(never parses logs)
detect	→ pure functions	(never reads the DB)
queries	→ read models	(never mutates)
DB.swift	→ the same read models	(read-only; the collector owns writes)
views	→ Store in, SwiftUI out	(never touch SQLite)

The database

Lives at `~/ .vole/vole.db` (WAL mode). Override with `VOLE_DB`.

It is **disposable** — delete it and re-run `pnpm collect` to rebuild from source logs.

```
sqlite3 ~/ .vole/vole.db

.tables
SELECT tool, confidence, COUNT(*), SUM(total_tokens) FROM usage_events GROUP BY 1,2;
SELECT source, rule, COUNT(*) FROM anomalies GROUP BY 1,2;
SELECT * FROM collector_state;
```

Useful resets:

```
rm -rf ~/ .vole          # full reset
sqlite3 ~/ .vole/vole.db "DELETE FROM collector_state;" # force full re-parse
sqlite3 ~/ .vole/vole.db "DELETE FROM anomalies;"      # re-run detection clean
```

Deleting `collector_state` is safe: `event_key` is `UNIQUE` and inserts are `INSERT OR IGNORE`, so a full re-parse cannot duplicate anything.

Testing against fixtures

`paths.ts` reads `VOLE_HOME_OVERRIDE`, so you can point collectors at a fixture tree:

```
VOLE_HOME_OVERRIDE=/tmp/fixture-home \  
VOLE_DB=/tmp/test.db \  
pnpm --filter @vole/core collect -- --once --verbose
```

Testing philosophy

Two layers, deliberately different in kind.

1. Unit tests — `pnpm test`

Rules and queries are pure or DB-only, so they are tested against fixture arrays and temp databases. 15 tests covering rule thresholds, bucket alignment, and confidence invariants.

Tests encode *intent*, not just behaviour. Example — a test that pins the boundary deliberately:

```
test('loop: exactly 3x the baseline does not fire (threshold is strictly greater)', ...)
```

2. Reconciliation — `pnpm verify`

This is the one that matters. It:

- re-implements the cost formula **independently** of `pricing.ts`, so a bug there cannot cancel itself out;
- compares **per message**, not by totals — totals drift continuously while a Claude Code session is live, so a total-vs-total check can neither prove nor disprove correctness;
- reports the live duplication factor.

```
duplication factor      2.11x  <- inflation avoided by dedup  
reconciled per-message  400  
cost mismatches         0  
worst cost delta (USD)  0.00e+0
```

PASS — every stored row matches its source record.

Run `pnpm verify` after any change to a collector or to pricing. If the PASS line changes, stop and investigate before doing anything else.

Debugging recipes

"A number looks wrong"

Work outward from the source, not inward from the UI.


```
# 1. Does the raw log agree?
cat ~/.claude/projects/*/*.jsonl | jq -r 'select(.message.usage!=null) | .message.id' | sort -u | wc -l

# 2. Does the DB agree with the raw log?
pnpm verify

# 3. Do the two read models agree with the DB?
pnpm --silent digest --range=all --json | head -20          # core's queries
swift run --package-path apps/mac Vole --dump              # the app's port of them
sqlite3 ~/.vole/vole.db "SELECT COUNT(*), SUM(total_tokens) FROM usage_events;"

# 4. Only then look at the view.
```

Nine times in ten the answer is at step 2 or 3.

"The app says 'Waiting for the collector' but data exists"

The app keys that state on `collector_state.last_scanned_at`, the collector's heartbeat. Check it, and check the app is reading the file you think it is (`$VOLE_DB` overrides the default):

```
sqlite3 ~/.vole/vole.db "SELECT MAX(last_scanned_at) FROM collector_state;"
swift run --package-path apps/mac Vole --dump      # prints the path it opened
```

If the collector was started against a different `VOLE_DB`, the two are looking at different files.

"Collector finds no files"

```
ls ~/.claude/projects/ ~/.codex/sessions/ 2>&1
sqlite3 ~/.vole/vole.db "SELECT tool, COUNT(*) FROM collector_state GROUP BY 1;"
```

Collectors fail in isolation — one broken source is reported in `notes` and does not stop the others. Run with `--verbose` to see those notes.

"A number in the app differs from `pnpm top` / `pnpm digest`"

The app charts `source = 'live'` rows only and never includes seed data; the CLIs do the same. If they still differ, `DB.swift` and `queries.ts` have drifted — diff the SQL, they are meant to be the same statement.

Conventions

- **Comments explain *why*, not *what*.** Nearly every comment in this codebase documents a constraint or a trap, because the tricky parts are all non-obvious.
- **NULL never becomes 0.** Unknown cost and absent tokens render as `-`.
- **Numbers in the app use `.monospacedDigit()`** so columns stay comparable.
- **Colours are the system palette**, plus one brand colour per tool in `Pal.series`; status colours are never reused as a series colour.
- **Match the surrounding code.** The codebase is small and consistent; follow the file you are in.

Next: [EXTENDING.md](#) to add a tool, a rule, or a panel.

Extending Vole

Step-by-step recipes with real code. Each one is a complete change: files to touch, code to write, and how to verify it worked.

- [Adding a collector \(a new tool\)](#)
- [Adding an anomaly rule](#)
- [Adding a model rate](#)
- [Adding a panel to the app](#)
- [Changing the schema](#)
- [Tuning rule thresholds](#)

Adding a collector

Say you want to support a tool called **Aider**.

Step 0 — Investigate before writing code

Do not skip this. The method is in [DATA-SOURCES.md](#) → [Investigating a new tool](#). You must answer one question first:

Does this tool record real token counts, a defensible proxy, or nothing?

That answer decides your `confidence` value, and everything downstream depends on it.

Step 1 — Register the tool

`packages/core/src/types.ts` :

```
export type Tool = 'claude_code' | 'codex' | 'cursor' | 'antigravity' | 'aider';
```

TypeScript will now flag every place that needs updating. Follow the errors.

Step 2 — Add the path

`packages/core/src/paths.ts` :

```
export const paths = {  
  // ...  
  /** Aider chat history: ~/.aider/history/*.jsonl */  
  aiderHistory: () => join(home(), '.aider', 'history'),  
};
```

Use the `home()` helper, never `homedir()` directly — that is what makes `VOLE_HOME_OVERRIDE` work for fixture-based tests. Follow the per-source override pattern too (`process.env.VOLE_<SOURCE> ?? join(home(), ...)`), so any install can point a single source at a non-standard location without touching code.

Step 3 — Write the collector

`packages/core/src/collectors/aider.ts` :

```

import { readdirSync, existsSync } from 'node:fs';
import { join } from 'node:path';
import { paths } from '../paths';
import { getState, setState, type DB } from '../db';
import { readNewLines, parseLine } from '../util/jsonl';
import { computeCost } from '../pricing';
import type { CollectorResult, UsageEvent } from '../types';

/**
 * Aider – <exact | estimated | activity_only>, because <the actual constraint you found>.
 */
export function collectAider(db: DB): CollectorResult {
  const root = paths.aiderHistory();
  const events: UsageEvent[] = [];
  const notes: string[] = [];
  let filesScanned = 0;

  // A missing directory is normal, not an error: the user may not have this tool.
  if (!existsSync(root)) {
    return { tool: 'aider', events, filesScanned, notes: ['No directory at ${root}'] };
  }

  for (const file of readdirSync(root)) {
    if (!file.endsWith('.jsonl')) continue;
    const filePath = join(root, file);
    filesScanned++;

    const state = getState(db, filePath);
    let result;
    try {
      result = readNewLines(filePath, state?.last_offset ?? 0);
    } catch (err) {
      notes.push(`Could not read ${file}: ${(err as Error).message}`);
      continue; // one bad file must not stop the rest
    }

    for (const line of result.lines) {
      const entry = parseLine<any>(line);
      if (!entry?.usage) continue;

      const tokens = {
        input_tokens: entry.usage.prompt_tokens ?? 0,
        output_tokens: entry.usage.completion_tokens ?? 0,
        cache_write_5m_tokens: 0,
        cache_write_1h_tokens: 0,
        cache_read_tokens: 0,
      };

      events.push({
        // Must be STABLE across re-reads – this is what makes polling idempotent.

```

```

        event_key: `aider:${entry.id}`,
        tool: 'aider',
        model: entry.model ?? null,
        session_id: entry.session ?? null,
        project: entry.cwd ?? null,
        git_branch: null,
        ts: Date.parse(entry.timestamp),
        ...tokens,
        reasoning_tokens: 0,
        total_tokens: tokens.input_tokens + tokens.output_tokens,
        cost_usd: computeCost(entry.model ?? null, tokens),
        confidence: 'exact',
        is_error: 0,
        stop_reason: null,
        source: 'live',          // collectors NEVER write 'seed'
        raw_ref: filePath,
    });
}

    setState(db, filePath, 'aider', result.newOffset, result.mtimeMs);
}

    return { tool: 'aider', events, filesScanned, notes };
}

```

Step 4 — Register it

packages/core/src/collectors/index.ts :

```

import { collectAider } from './aider';
export { collectAider };

const REGISTRY: { tool: Tool; run: (db: DB) => CollectorResult }[] = [
    // ...
    { tool: 'aider', run: collectAider },
];

```

Step 5 — Add UI identity

apps/mac/Sources/Vole/Theme.swift :

```

// Pal.series — one brand colour per tool, never reused for status.
case "aider": return Color(red: 0.13, green: 0.59, blue: 0.95)
// Pal.symbol — SF Symbol fallback when no logo is bundled.
case "aider": return "terminal"
// Labels.order / Labels.tool / Labels.toolShort
"aider": "Aider"

```

Drop a `aider.png` into `Sources/Vole/Resources/` and `ToolIcon` picks it up automatically; add the name to the `monochrome` set if the mark is near-black. The CLIs need nothing — they print the tool id — but `packages/core/src/digest.ts` and `cli/top.ts` carry a label map worth extending.

Step 6 — Verify

```
pnpm collect:once
```

Check three things:

1. Your tool appears with a plausible call count.
2. Re-run it — `dedup-skipped` should equal `parsed`, and `new` should be `0`. If not, your `event_key` is not stable.
3. `confidence` is what you decided in Step 0.

```
sqlite3 ~/.vole/vole.db \  
  "SELECT confidence, COUNT(*), SUM(total_tokens IS NULL) FROM usage_events WHERE tool='aider' GROUP BY  
  1;"
```

If you chose `activity_only`, every token column must be `NULL` — the invariant test in `queries.test.ts` covers this pattern.

Step 7 — Document it

Add a section to [DATA-SOURCES.md](#) and a row to the README table. **This is not optional.** The project's credibility rests on every number being traceable to a documented source.

Adding an anomaly rule

Rules are pure functions: `(events, now) => Anomaly[]`. No DB access, so they are trivially testable.

Step 1 — Name it

```
packages/core/src/types.ts :
```

```
export type AnomalyRule =  
  | 'burn_rate_spike' | 'loop_suspected' | 'error_storm' | 'rate_limit_pressure'  
  | 'context_thrash';
```

Step 2 — Write it

```
packages/core/src/detect/context-thrash.ts :
```

```

import type { Anomaly, UsageEvent } from '../types';
import { bucketOf, groupBy, medianExcluding, withTokens, worstConfidence, fmt, shortId } from './util';

const WINDOW_MS = 15 * 60 * 1000;
const MIN_CALLS = 10;
const RATIO_THRESHOLD = 0.97; // cache reads this dominant means almost no new work

/**
 * Context thrash: the session is re-reading an enormous cached context to produce
 * almost nothing new. Distinct from a loop, which is about call *frequency*.
 */
export function detectContextThrash(events: UsageEvent[], now: number): Anomaly[] {
  const out: Anomaly[] = [];

  // withTokens() drops activity_only rows – they have no tokens to reason about.
  for (const [sessionId, group] of groupBy(withTokens(events), (e) => e.session_id ?? 'none')) {
    const windows = groupBy(group, (e) => String(bucketOf(e.ts, WINDOW_MS)));

    for (const [bucketStr, evs] of windows) {
      if (evs.length < MIN_CALLS) continue;

      const read = evs.reduce((s, e) => s + (e.cache_read_tokens ?? 0), 0);
      const total = evs.reduce((s, e) => s + (e.total_tokens ?? 0), 0);
      if (total === 0) continue;

      const ratio = read / total;
      if (ratio < RATIO_THRESHOLD) continue;

      const first = evs[0];
      if (!first) continue;
      const bucket = Number(bucketStr);

      out.push({
        // Stable key: rule + tool + session + window. Never include `now`,
        // or every poll creates a duplicate incident.
        anomaly_key: `context_thrash:${first.tool}:${sessionId}:${bucket}`,
        rule: 'context_thrash',
        severity: ratio > 0.99 ? 'critical' : 'warn',
        tool: first.tool,
        session_id: sessionId,
        model: first.model,
        window_start: bucket,
        window_end: bucket + WINDOW_MS,
        title: `Context thrash in ${first.tool} session ${shortId(sessionId)}`,
        // Put the real numbers in the text. An incident the reader cannot check is noise.
        detail:
          `${(ratio * 100).toFixed(1)}% of ${fmt(total)} tokens in 15 min were cached re-reads ` +
          `across ${evs.length} calls. The session is re-processing context rather than ` +
          `producing new work.`,
        observed: ratio,
      });
    }
  }
}

```

```

        baseline: RATIO_THRESHOLD,
        threshold: RATIO_THRESHOLD,
        confidence: worstConfidence(evs), // only as trustworthy as its weakest input
        source: first.source,
        detected_at: now,
    });
}
}
return out;
}

```

Step 3 — Register it

`packages/core/src/detect/index.ts` :

```

import { detectContextThrash } from './context-thrash';
export { detectContextThrash };

export function detectAll(events, rateLimits = [], now = Date.now()): Anomaly[] {
    return [
        // ...
        ...detectContextThrash(events, now),
    ];
}

```

`detectBySource()` picks it up automatically — it wraps `detectAll` .

Step 4 — Label it in the UI

`apps/mac/Sources/Vole/Theme.swift` , `Labels.rule` , and the `RULE` map in `packages/core/src/digest.ts` :

```

"context_thrash": "Context thrash",

```

Step 5 — Test both directions

The negative test matters as much as the positive one. A rule that fires on everything is worse than no rule.

```

test('context thrash: healthy cache use does not fire', () => {
  const events = Array.from({ length: 20 }, (_, i) =>
    ev({ ts: T0 + i * 30_000, cache_read_tokens: 50_000, total_tokens: 100_000 }));
  assert.equal(detectContextThrash(events, T0).length, 0);
});

test('context thrash: near-total cached re-reads fires', () => {
  const events = Array.from({ length: 20 }, (_, i) =>
    ev({ ts: T0 + i * 30_000, cache_read_tokens: 99_500, total_tokens: 100_000, output_tokens: 40 }));
  const found = detectContextThrash(events, T0);
  assert.equal(found.length, 1);
});

test('context thrash: keys are stable across runs', () => {
  const a = detectContextThrash(events, T0);
  const b = detectContextThrash(events, T0 + 999_999); // different "now"
  assert.deepEqual(a.map(x => x.anomaly_key), b.map(x => x.anomaly_key));
});

```

Step 6 — Exercise it in the seeder

Real logs may never trigger your rule. Add a block to `packages/core/src/cli/seed.ts` that deliberately produces the pattern, so a demo shows it. Follow the existing blocks.

Step 7 — Verify end to end

```

pnpm test
pnpm seed:purge && pnpm seed
sqlite3 ~/.vole/vole.db "SELECT source, rule, COUNT(*) FROM anomalies GROUP BY 1,2;"

```

Run `pnpm collect:once` twice — the anomaly count must not grow. If it does, your `anomaly_key` is unstable.

Adding a model rate

Rates are data, not code: `packages/core/src/data/pricing.json` :

```

{
  "models": {
    "gpt-5.1-codex-max": { "input": 1.25, "output": 10.0, "effective_from": "2026-08-30" }
  }
}

```

Then remove it from the `unpriced` map in the same file if listed. New rows are priced at once, and rows stored before the rate existed are priced the next time the collector starts (`repriceUnpriced()` in `db.ts`) — no manual backfill.

Cache multipliers are model-independent by default (read 0.1x, 5m write 1.25x, 1h write 2x). A model whose provider prices cache reads differently sets a flat `"cache_read": <$/MTok>` on its entry, which replaces the read multiplier for that model only. Dated snapshot ids (`claude-haiku-4-5-20251001`) resolve to their alias's rate, so only the alias needs an entry.

The same file's `context_windows` map gives each model's window in tokens; it drives the `context_pressure` rule and the context gauge in `pnpm top` / the MCP live-session view. Add a model there only when its window is published — an unknown window is skipped, never guessed.

A per-installation override (`~/vole/pricing.json` , or `$VOLE_PRICING`) merges over the built-in table — add rates there without touching the repo.

Always run `pnpm verify` afterwards. It re-implements the formula independently and will catch a mistyped rate. If you add a rate to `pricing.json` , mirror it in the `RATES` table at the top of `packages/core/src/cli/verify.ts` — that copy is deliberate, so a typo in the product's rate is caught instead of cancelling out.

Adding a panel to the app

A panel is a read model in core (so it is tested, and reachable from the CLIs and MCP), the same query ported to Swift, and a view.

Step 1 — Query in core

`packages/core/src/queries.ts` :

```
export interface SessionRow {
  session_id: string; tool: Tool; calls: number; tokens: number; started: number;
}

export function getTopSessions(db: DB, range: Range, includeSeed: boolean): SessionRow[] {
  const from = rangeStart(range);
  const seedClause = includeSeed ? '' : " AND source = 'live'";
  return db.prepare(
    `SELECT session_id, tool, COUNT(*) AS calls,
      COALESCE(SUM(CASE WHEN confidence != 'activity_only' THEN total_tokens END), 0) AS tokens,
      MIN(ts) AS started
    FROM usage_events
    WHERE ts >= ? AND session_id IS NOT NULL${seedClause}
    GROUP BY session_id ORDER BY tokens DESC LIMIT 10`
  ).all(from) as SessionRow[];
}
```

Note the `confidence != 'activity_only'` guard. **Every** token aggregate needs it, or tools that record no tokens quietly distort the result.

Add a test in `queries.test.ts` , and expose it as an MCP tool in `cli/mcp.ts` if an agent could use it.

Step 2 — Port it to Swift

`apps/mac/Sources/Vole/DB.swift` — the same SQL, character for character where possible, on the existing `run(_:_:_)` helper:

```

struct SessionRow: Identifiable {
    let sessionID: String, tool: String, calls: Int, tokens: Int, started: Int
    var id: String { sessionID }
}

func topSessions(_ r: DateRange) -> [SessionRow] {
    var out: [SessionRow] = []
    run("""
        SELECT session_id, tool, COUNT(*),
               COALESCE(SUM(CASE WHEN \((tf) THEN total_tokens END), 0), MIN(ts)
        FROM usage_events WHERE ts >= ? AND session_id IS NOT NULL AND source = 'live'
        GROUP BY session_id ORDER BY 4 DESC LIMIT 10
        """, [r.startMs()]) { row in
        out.append(SessionRow(sessionID: colText(row, 0) ?? "", tool: colText(row, 1) ?? "",
                               calls: colInt(row, 2), tokens: colInt(row, 3), started: colInt(row, 4)))
    }
    return out
}

```

Then add a `topSessions` property to `Store.swift` and set it inside `refresh()` .

Step 3 — View

`apps/mac/Sources/Vole/DashboardView.swift` — a `Section` in the relevant pane, standard controls, nothing custom:

```

Section("Top sessions") {
    ForEach(store.topSessions) { s in
        LabeledContent {
            Text(Fmt.compact(s.tokens)).monospacedDigit()
        } label: {
            Label { Text(s.sessionID.prefix(8)) } icon: { ToolIcon(tool: s.tool, size: 14) }
        }
    }
}

```

UI checklist

- Numbers use `.monospacedDigit()` so columns stay comparable.
- `NULL` renders as `–` via `Fmt.compact / Fmt.money` , never `0` .
- Non-exact data carries `ConfidenceBadge` .
- Colours are the system palette or `Pal.series(tool)` ; status colours stay reserved.
- Empty state says what to do (see `SetupCard`), not just "no data".
- Run `swift run Vole --dump` to check the new query against the real file before wiring the view.

Changing the schema

The database is disposable — it rebuilds from source logs — so there is no full migration system. `openDb()` in `db.ts` does run a tiny `migrate()` that **adds new nullable columns** to existing databases (that is how

user / machine landed), so the common case is painless. Anything more (renaming, re-typing, backfilling) still requires a rebuild.

1. Edit `SCHEMA` in `packages/core/src/schema.ts`.
2. Update `UsageEvent` / `Anomaly` in `types.ts`.
3. Update the `INSERT` statements in `db.ts` (both column list and `@named` params).
4. New nullable column? Add it to `migrate()` and existing databases pick it up automatically. Anything else, rebuild:

```
rm ~/.vole/vole.db*
pnpm collect:once && pnpm seed
```

If you need real migrations later, see [ROADMAP.md](#).

Tuning rule thresholds

Every threshold is a named constant at the top of its rule file:

```
const WINDOW_MS = 10 * 60 * 1000;
const MIN_TOKENS_IN_WINDOW = 20_000; // floor: below this a "spike" is noise
const SPIKE_MULTIPLE = 3;
```

To see the effect of a change:

```
sqlite3 ~/.vole/vole.db "DELETE FROM anomalies;"
pnpm collect:once
sqlite3 ~/.vole/vole.db "SELECT rule, severity, COUNT(*) FROM anomalies GROUP BY 1,2;"
```

Two principles worth preserving:

Keep the absolute floor. Ratios alone fire on trivial data — 3 errors out of 4 calls is a quiet session, not an incident.

Keep baselines leave-one-out. `medianExcluding()` stops an outlier inflating the baseline it is measured against. Reverting to a plain median makes lone spikes invisible, which is the exact failure this project was built to catch.

Roadmap

Concrete ways to take this further, ranked within each tier by value-to-effort. Each entry names the files involved so you can start immediately.

Effort is rough: **S** = an afternoon · **M** = a day or two · **L** = a week or more.

Tier 1 — Close the known gaps

These are documented weaknesses. Fixing them makes existing claims fully true.

Correlate tool errors back to their assistant message · M

Problem: `is_error` only captures API errors (`isApiErrorMessage`). Failed *tool results* inside a turn are not linked to the assistant message that issued them, so `error_storm` under-counts on live data — it currently fires reliably only on seeded data.

Approach: while parsing a Claude Code transcript, buffer `tool_use` block ids from assistant messages, then scan subsequent `type: "user"` entries whose content contains `tool_result` blocks with `is_error: true` . Map the result back via `tool_use_id` and set `is_error = 1` on the originating event.

Files: `packages/core/src/collectors/claude-code.ts` , plus a fixture test.

Watch out: the tool result may arrive after the assistant event has already been inserted. Either buffer within the file pass, or add a second `UPDATE` pass keyed on `event_key` .

Signed, notarised app · S–M

Problem: `bundle.sh` ad-hoc signs `Vole.app` ; it runs locally but cannot be distributed.

Approach: an Apple Developer ID, `codesign --options runtime` , `notarytool` , and a `.dmg` . `bundle.sh` already assembles the bundle, so this is the last mile. A Homebrew cask after that.

Collector as a sidecar · M

You still start `pnpm collect` yourself. The app could launch and supervise it (a `Process` with the bundled Node, or a `launchd` agent installed from Settings), so the whole product is one icon.

Files: `apps/mac/Sources/Vole/Store.swift` , `VoleApp.swift` .

Tier 2 — Make it a better monitor

Port the core-only views into the app · M

The highest-value item on this list. These are complete and tested in `packages/core` , and reachable from `pnpm top` , `pnpm digest` and the MCP server, but the app does not draw them yet:

- **Live sessions** — `getLiveSessions()` : context vs window gauge, tokens/min, last tools, the prompt-cache countdown and re-warm cost, open incidents.
- **Session drill-down** — `getSessionDetail()` : context per call as one line per agent thread, the subagent cost tree, "what grew the context" (the largest jumps and the tools whose results came back before them). Link from every incident.
- **Digest** — `getDigest()` / `digestMarkdown()` : the "your agent week" card with a share sheet.
- **What-if** — `getWhatIf()` : the same token split at other models' list rates, labelled as arithmetic only.

Each is a `DB.swift` port of one query plus a SwiftUI section; see [EXTENDING.md → Adding a panel to the app](#).

Budget thresholds and a burn-down · M

Let the user set a daily or monthly token/cost budget and show pace against it — "at this rate you will hit your budget on the 22nd".

Approach: a small `settings` table or a config file, a `budget_exceeded` rule, and a `Gauge` in the app. A single ratio against a limit is a meter, not a pie.

Files: new `packages/core/src/detect/budget.ts`, new settings storage, a new metric row.

Incident deep links · S

An incident names a session; the app should open that session's drill-down (above) from the incident row, and the desktop notification should open the app to it.

Incident acknowledgement · S

Add `acknowledged_at` to `anomalies` so handled incidents can be dismissed. Without it the feed becomes noise over time.

Note: requires a schema change — see [EXTENDING.md → Changing the schema](#).

Tier 3 — Broaden coverage

More tools · M each

Gemini CLI, GitHub Copilot CLI (`~/copilot/` exists on many machines), Aider, Cline, Continue. Follow [EXTENDING.md → Adding a collector](#), and do the investigation step first — the whole point is to establish honestly what each one records.

Cross-platform paths · S

`paths.ts` assumes macOS/Linux layouts. Windows needs `%APPDATA%` variants. The `home()` indirection already exists, so this is contained.

Import Claude Code's own cost data · S

Some Claude Code versions record additional cost metadata. Worth re-checking the transcript schema periodically and cross-verifying against the computed figure — a second independent source would strengthen `pnpm verify`.

Tier 4 — Bigger swings

Multi-machine aggregation · L

The premise is local-only, so this is a genuine architectural change. If you do it, keep collection local and sync only the normalised `usage_events` rows — never raw transcripts, which contain source code and prompts.

Warning: this crosses a privacy line the project currently guarantees. Make it opt-in, explicit, and documented.

Anomaly detection that learns · L

Current rules are fixed-threshold and explainable, which is a feature: every incident says exactly why it fired. Any statistical model must preserve that. A reasonable middle path is per-session seasonal baselines (weekday/hour aware) rather than a global median.

Do not replace explainable rules with an opaque score. "Possible runaway loop — 31 calls in 5 min while output stayed at 262 tokens" is the product.

Live agent control · L

The logical endpoint of a sentry: not just detecting a runaway loop but offering to stop it. Would require process discovery and a kill/pause mechanism per tool. High value, high risk — a false positive kills real work.

Historical rollups · M

`usage_events` grows without bound. Add a monthly rollup table and prune raw rows beyond a retention window. Not urgent — SQLite handles millions of rows fine — but eventually needed.

Good first issues

If you are new to the codebase, these are self-contained:

1. **Keyboard shortcuts for the range picker** in the app (`1 / 7 / 3 / a`).
 2. A `--json` flag on `collect` so the summary can be piped into other tools.
 3. **Export the breakdown table as CSV** from the app (a share/save button).
 4. **Launch at login** — the Settings toggle exists; wire it to `SMApService` .
 5. A `--range=` flag on `pnpm top` and colour for the context gauge in the terminal.
 6. **Register the MCP server** from the app's Settings pane (writes the config for each tool).
-

Principles to preserve

Whatever you build, these are what make the project defensible. Breaking them costs more than any feature gains.

1. **Never invent a number.** If a tool does not record it, the field is `NULL` and the UI says so.
2. **Keep `exact` / `estimated` / `activity_only` visible** wherever a number is shown.
3. **Keep incidents explainable.** Every anomaly carries the real figures that triggered it.
4. **Keep `pnpm verify passing`**, and extend it when you add a source with exact data.
5. **Keep collection idempotent.** Stable `event_key` s; re-running must never double-count.
6. **Keep seed and live data separate** — in storage *and* in statistical baselines.
7. **Keep it local.** No scraping, no web-UI automation, no silent network calls.